Submit a feed

Learn how to submit a feed, check the status of feed processing, and verify that your feed submission was successful.

> **!** Important
>
> Starting **July 31, 2025**, the Feeds API no longer supports legacy XML and flat file listing feeds, including pricing, inventory, relationships, and images. Developers will receive fatal `processingStatus` in the `getFeed` API response for these feeds. For more information, refer to the deprecation announcement.

Learn how to submit a feed, check the status of feed processing, and verify that your feed submission was successful. The tutorial contains Java code samples that demonstrate a way to upload a feed and download a feed processing summary report. You can use the principles demonstrated in the sample code to guide you in building applications in other programming languages, using other HttpClient libraries or upload feeds with different formats.

Prerequisites

To complete this tutorial, you need:

- A feed to submit. Refer to Feed Type Values for a list of available feed types.
- Authorization from the seller for whom you are making calls. Refer to Authorizing Selling Partner API applications for more information.
- A working Java Development Kit (JDK) installation.

Step 1. Create a feed document

1. Call the `createFeedDocument` operation.
2. Save the following values:
    1. `url`. Use this value in Step 3. Upload the feed data.

2. `feedDocumentId`. Use this value in Step 4. Create a feed. This `feedDocumentId` value expires after two days. If you pass in an expired `feedDocumentId` value to the `createFeed` operation, the call will fail.

Step 2. Construct a feed

Construct a feed that you can upload in Step 3. Upload the feed data.

XML feeds

To construct an XML feed you need to include the three core XSDs (Base, Envelope, and Header) plus your category-specific feed.

Core XSDs:

- **Base.** Used to promote consistency among feeds. All other XSDs reference the elements and data types in the Base XSD.
- **Envelope.** Used to wrap all other data with message-level protocol data. Consists of a header and one or more messages.
- **Header.** Used by the Envelope XSD to specify universal data related to the feed or a message in the feed.

For links to XSDs for category-specific feeds, go to XSDs in the Seller Central Help and look in the **Category XSDs** section.

The following is an example of an XML feed for a health-related product:

> **Use a Seller ID for `MerchantIdentifier`**
>
> The value of `MerchantIdentifier` in the following feed must be a Seller ID. Your Seller ID is located in Seller Central under **Settings** > **Account Info** > **Your Merchant Token**.

XML

```
<?xml version="1.0" encoding="iso-8859-1"?>
<AmazonEnvelope
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:noNamespaceSchemaLocation="amzn-envelope.xsd">
  <Header>
    <DocumentVersion>1.01</DocumentVersion>

<MerchantIdentifier>XXXXXXXXXXXXX</MerchantIdentifier>
  </Header>
  <MessageType>Product</MessageType>
  <PurgeAndReplace>false</PurgeAndReplace>
  <Message>
    <MessageID>1</MessageID>
    <OperationType>Update</OperationType>
    <Product>
      <SKU>56789</SKU>
      <StandardProductID>
        <Type>ASIN</Type>
        <Value>B0EXAMPLEG</Value>
```

Special Authorization Types

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

```
      </StandardProductID>
      <ProductTaxCode>A_GEN_NOTAX</ProductTaxCode>
      <DescriptionData>
        <Title>Example Product Title</Title>
        <Brand>Example Product Brand</Brand>
        <Description>This is an example product
description.</Description>
        <BulletPoint>Example Bullet Point
1</BulletPoint>
        <BulletPoint>Example Bullet Point
2</BulletPoint>
        <MSRP currency="USD">25.19</MSRP>
        <Manufacturer>Example Product
Manufacturer</Manufacturer>
        <ItemType>example-item-type</ItemType>
      </DescriptionData>
      <ProductData>
        <Health>
          <ProductType>
            <HealthMisc>
              <Ingredients>Example
Ingredients</Ingredients>
              <Directions>Example
Directions</Directions>
            </HealthMisc>
          </ProductType>
        </Health>
      </ProductData>
    </Product>
  </Message>
</AmazonEnvelope>
```

JSON feeds

The following is an example of a JSON feed:

JSON

```
{
  "header": {
    "sellerId": "AXXXXXXXXXXXX",
    "version": "2.0",
    "issueLocale": "en_US"
  },
  "messages": [
    {
      "messageId": 1,
      "sku": "My-SKU-A",
      "operationType": "DELETE"
    },
    {
      "messageId": 2,
      "sku": "My-SKU-B",
      "operationType": "PARTIAL_UPDATE",
      "productType": "LUGGAGE",
      "attributes": {
        "fulfillment_availability": [
          {
            "fulfillment_channel_code": "DEFAULT",
            "quantity": 10
          }
        ]
```

```
    }
  },
  {
    "messageId": 3,
    "sku": "My-SKU-C",
    "operationType": "UPDATE",
    "productType": "LUGGAGE",
    "requirements": "LISTING_OFFER_ONLY",
    "attributes": {
      "condition_type": [
        {
          "value": "new_new",
          "marketplace_id": "ATVPDKIKX0DER"
        }
      ],
      "merchant_suggested_asin": [
        {
          "value": "AXXXXXXXXXXXX",
          "marketplace_id": "ATVPDKIKX0DER"
        }
      ],
      "fulfillment_availability": [
        {
          "fulfillment_channel_code": "DEFAULT",
          "quantity": 0
        }
      ],
      "purchasable_offer": [
        {
          "currency": "USD",
          "our_price": [
            {
              "schedule": [
                {
                  "value_with_tax": 100
                }
              ]
            }
          ],
          "marketplace_id": "ATVPDKIKX0DER"
        }
      ]
    }
  },
  {
    "messageId": 4,
    "sku": "My-SKU-D",
    "operationType": "PATCH",
    "productType": "LUGGAGE",
    "patches": [
      {
        "op": "replace",
        "path":
```

Step 3. Upload the feed data

You can upload the feed that you constructed in Step 2. Construct a feed using the information returned in Step 1. Create a feed document. The following sample code demonstrates one way to upload the feed content. You can also use the principles demonstrated in the sample code to guide you in building applications in other programming languages or using other HttpClient libraries.

The sample `upload` method shown in the `UploadExample` class accepts your feed content as the first argument, and the `url` value that you saved in Step 1 as the second argument.

Sample code to upload an XML feed (Java)

Java

```
// UploadExample.java
// This example is for use with the Selling Partner
API for Feeds, Version: 2021-06-30
import java.io.IOException;
import java.nio.charset.StandardCharsets;

import com.squareup.okhttp.MediaType;
import com.squareup.okhttp.OkHttpClient;
import com.squareup.okhttp.Request;
```

Edit Your Appstore Listing
Check Listing Status
Appstore Ratings and Reviews
Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview
Amazon Selling Partner API Guard Implementation Guide
VAT Calculation Service
Amazon Seller Data Access
Best Practices

```
import com.squareup.okhttp.RequestBody;
import com.squareup.okhttp.Response;

/**
 * Example that uploads content.
 */
public class UploadExample {

  public static void main(String args[]) {
    String url = "<URL from the createFeedDocument
operation>";
    String content = "<your feed content>";

    UploadExample obj = new UploadExample();

obj.upload(content.getBytes(StandardCharsets.UTF_8),
url);
  }

  /**
   * Upload content to the given URL.
   *
   * @param source the content to upload
   * @param url    the URL to upload content
   */
  public void upload(byte[] source, String url) {
    OkHttpClient client = new OkHttpClient();

    // The contentType must match the input provided
to the createFeedDocument operation. This example uses
text/xml, but your contentType may be different
depending upon on your chosen feedType (text/plain,
text/csv, and so on).
    String contentType = String.format("text/xml;
charset=%s", StandardCharsets.UTF_8);
    try {
      Request request = new Request.Builder()
        .url(url)

.put(RequestBody.create(MediaType.parse(contentType),
source))
        .build();

      Response response =
client.newCall(request).execute();
      if (!response.isSuccessful()) {
        System.out.println(
          String.format("Call to upload document
failed with response code: %d and message: %s",
            response.code(), response.message()));
      }
    } catch (IOException e) {
      System.out.println(e.getMessage());
```

Sample code to upload a JSON feed (Java)

Java

```
// UploadExample.java
// This example is for use with the Selling Partner
API for Feeds, Version: 2021-06-30
import java.io.IOException;
import java.nio.charset.StandardCharsets;

import com.squareup.okhttp.MediaType;
import com.squareup.okhttp.OkHttpClient;
import com.squareup.okhttp.Request;
```

A+ CONTENT API

A+ Content API

A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API

Amazon Warehousing and Distribution API

Amazon Warehousing and Distribution API v2024-05-09 Reference

APP INTEGRATIONS API

App Integrations API

APPLICATION MANAGEMENT API

Application Management API

CATALOG ITEMS API

Catalog Items API

Catalog Items API Rate Limits

Catalog Items API v2022-04-01 Reference

CUSTOMER FEEDBACK API

Customer Feedback API

Customer Feedback API Rate Limits

```
import com.squareup.okhttp.RequestBody;
import com.squareup.okhttp.Response;

/**
 * Example that uploads content.
 */
public class UploadExample {

  public static void main(String args[]) {
    String url = "<URL from the createFeedDocument
operation>";
    String jsonData = "<your JSON data>";

    UploadExample obj = new UploadExample();

obj.upload(jsonData.getBytes(StandardCharsets.UTF_8),
url);
  }

  /**
   * Upload content to the given URL.
   *
   * @param source the content to upload
   * @param url    the URL to upload content
   */
  public void upload(byte[] source, String url) {
    OkHttpClient client = new OkHttpClient();

    // The contentType must match the input provided
to the createFeedDocument operation. This example uses
application/json, but your contentType may be
different depending on your chosen feedType
(text/plain, text/csv).
    String contentType =
String.format("application/json; charset=%s",
StandardCharsets.UTF_8);
    try {
      Request request = new Request.Builder()
        .url(url)

.put(RequestBody.create(MediaType.parse(contentType),
source))
        .build();

      Response response =
client.newCall(request).execute();
      if (!response.isSuccessful()) {
        System.out.println(
          String.format("Call to upload document
failed with response code: %d and message: %s",
            response.code(), response.message()));
      }
    } catch (IOException e) {
      System.out.println(e.getMessage());
    }
  }
}
```

Step 4. Create a feed

1. Call the `createFeed` operation.
2. Save the `feedId` value. Pass this value in the `getFeed` operation in Step 5. Confirm feed processing.

Step 5. Confirm feed processing

Confirm feed processing by periodically calling the `getFeed` operation until the feed moves into one of the following terminal states: `DONE`, `CANCELLED`, or `FATAL`. When the feed moves into the `DONE` state, proceed to Step 6. Get information for retrieving the feed processing report.

DATA KIOSK

Data Kiosk API

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API

EASY SHIP API

Easy Ship API

EXTERNAL FULFILLMENT

External Fulfillment Inventory API

External Fulfillment Returns API

External Fulfillment Shipping API

External Fulfillment Errors

External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API

FBA Inventory Dynamic Sandbox Guide

FEEDS API

> **Feeds can take up to eight hours to process**
>
> Under high load conditions it is not uncommon for feeds to take up to eight hours to process. Product data feeds are processed sequentially; the most recent feed will be queued in the processing system until previous feed submissions have completed. Substantial processing delays can occur when multiple product feeds contain only a few items each instead of a single product feed with all items.

1. Call the `getFeed` operation.
2. Check the value of the `processingStatus` attribute.
    1. If `processingStatus` is `IN_QUEUE` or `IN_PROGRESS`, feed processing is not yet complete. Retry the `getFeed` operation until `processingStatus` reaches one of the following terminal states: `DONE`, `CANCELLED`, or `FATAL`.
    2. If `processingStatus` is `DONE`, feed processing is complete. Go to Step 6. Get information for retrieving the feed processing report.
    3. If `processingStatus` is `CANCELLED`, the feed was cancelled before it started processing. If you want to submit the feed again, start again at Step 1. Create a feed document.
    4. If `processingStatus` is `FATAL`, the feed was stopped due to a fatal error. Some, none, or all of the operations within the feed might have completed successfully. In some (but not all) cases Amazon generates a feed processing report. If Amazon generates a report, it could be in a different format from a feed processing report for a successfully completed feed. Go to Step 6. Get information for retrieving the feed processing report to attempt to retrieve a feed processing report. In rare cases Amazon might stop a feed for reasons unrelated to the feed. If you can find no errors in the feed to correct, try submitting the feed again.

> **Note**
>
> The `getFeed` operation only serves information for feed requests that were created within the last 28 days.

Step 6. Get information for retrieving the feed processing report

The feed processing report indicates which records in the feed that you submitted were successful and which records generated errors. In this step you get a presigned URL for downloading the feed processing report as well as the information required to decrypt the document's contents. In some cases the object can be compressed, in

which case `compressionAlgorithm` is returned in addition to the URL. The URL expires after five minutes.

1. Call the `getFeedDocument` operation.
2. Save the `url` and optional `compressionAlgorithm` values to pass in Step 7. Download the feed processing report.

Step 7. Download the feed processing report

You can download the feed processing report using the information returned in the previous step. The following Java sample code can help. You can also use the principles demonstrated in the Java sample code to guide you in building applications in other programming languages.

1. Use the following as inputs for the sample code:
    1. The `url` and optional `compressionAlgorithm` values from the previous step are arguments for the `url` and `compressionAlgorithm` parameters of the `download` method of the `DownloadExample` class.

> **Note**
>
> It is your responsibility to always maintain encryption at rest. Unencrypted feed processing report content should never be stored on disk, even temporarily, because feed processing reports can contain sensitive information. The sample code that we provide demonstrates this principle.

Download sample code (Java)

Java

```
// DownloadExample.java
// This example is for use with the Selling Partner
API for Reports, Version: 2021-06-30
// and the Selling Partner API for Feeds, Version:
2021-06-30
import java.io.BufferedReader;
import java.io.Closeable;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.nio.charset.Charset;
import java.util.zip.GZIPInputStream;

import com.squareup.okhttp.MediaType;
import com.squareup.okhttp.OkHttpClient;
import com.squareup.okhttp.Request;
import com.squareup.okhttp.Response;
import com.squareup.okhttp.ResponseBody;

/**
 * Example that downloads a document.
 */
public class DownloadExample {
```

```
  public static void main(String args[]) {
    String url = "<URL from the
getFeedDocument/getReportDocument response>";
    String compressionAlgorithm = "
<compressionAlgorithm from the
getFeedDocument/getReportDocument response>";

    DownloadExample obj = new DownloadExample();
    try {
      obj.download(url, compressionAlgorithm);
    } catch (IOException e) {
      //Handle exception here.
    } catch (IllegalArgumentException e) {
      //Handle exception here.
    }
  }

  /**
   * Download and optionally decompress the document
retrieved from the given url.
   *
   * @param url                 the url pointing to a
document
   * @param compressionAlgorithm the
compressionAlgorithm used for the document
   * @throws IOException              when there is an
error reading the response
   * @throws IllegalArgumentException when the charset
is missing
   */
  public void download(String url, String
compressionAlgorithm) throws IOException,
IllegalArgumentException {
    OkHttpClient httpclient = new OkHttpClient();
    Request request = new Request.Builder()
        .url(url)
        .get()
        .build();
    Response response =
httpclient.newCall(request).execute();
    if (!response.isSuccessful()) {
      System.out.println(
        String.format("Call to download content was
unsuccessful. HTTP response code: %d and message: %s",
          response.code(), response.message()));
      return;
    }
    try (ResponseBody responseBody = response.body())
{
      MediaType mediaType =
MediaType.parse(response.header("Content-Type"));
      Charset charset = mediaType.charset();
      if (charset == null) {
        throw new
IllegalArgumentException(String.format(
            "Could not parse character set from '%s'",
mediaType.toString()));
      }
      Closeable closeThis = null;
```

Step 8. Check the feed processing report for errors

Check the feed processing report for errors generated during processing. If there are no errors, your feed submission is complete. If there are errors, correct them and submit the corrected feed, starting at Step 1. Create a feed document. Repeat the process until there are no errors in the feed processing report.

Using multiple marketplaces

> **Note**
>
> The JSON Listings Feed (`JSON_LISTINGS_FEED`) only supports a single marketplace per feed. When using the `JSON_LISTINGS_FEED` , you must send a feed for each marketplace.

If you are registered in multiple marketplaces, then you have multiple `marketplaceId` values associated with your seller ID. For a list of `marketplaceId` values, refer to Marketplace IDs. You can submit a feed that is applied to one or several `marketplaceId` values. If you are in the EU or NA region, you can submit feeds to

Fulfillment Outbound API Rate Limits

MCF Best Practices

INVOICING

Invoices API

Invoices API FAQ

LISTINGS

Listings Items API

Listings Items API Rate Limits
Listings Restrictions API
Listings Restrictions API Rate Limits
Manage Product Listings with the Selling Partner API
Building Listings Management Workflows Guide
Understanding Amazon listing status and seller-fulfilled inventory management
Listings Management Workflow Migration
Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases
Listings APIs FAQ
Product Type Definitions API
Product Type Definitions API Rate Limits
MERCHANT FULFILLMENT API
Merchant Fulfillment API
MESSAGING API
Messaging API

support multiple marketplaces if you have registered using a single, unified seller account.

If you sell in multiple marketplaces, you can manage your inventory levels using the same SKUs across multiple marketplaces. This eliminates the need for you to manually keep inventory levels across several marketplaces in sync. For more information on selling in multiple marketplaces, refer to Selling on Amazon's European Marketplaces.

Understanding how `marketplaceId` values are used

You specify what marketplaces you want a feed to be applied to by supplying a list of `marketplaceId` values to the `marketplaceIds` parameter when you call the `createFeed` operation. For example, an EU multiple-marketplace seller could specify that a feed be applied to both their FR and DE marketplaces by specifying the `marketplaceIds` parameter as follows:

JSON

```
[
  "A13V1IB3VIYZZH",
  "A1PA6795UKMFR9"
]
```

For EU and NA sellers, you do not need to use a specific country endpoint, such as `https://sellingpartnerapi-eu.amazon.com`, to indicate what marketplace a feed is to be applied to. You can apply changes to a given EU or NA marketplace by specifying that `marketplaceId` when submitting a feed. For a list of `marketplaceId` values, refer to Marketplace IDs.

Flat File Order Adjustments Feed

Sellers can use the Flat File Order Adjustments Feed (`POST_FLAT_FILE_PAYMENT_ADJUSTMENT_DATA`) to update Amazon's system with order adjustment information. The feed allows you to issue a full refund (adjustment) for an order. You must provide a reason for the adjustment, such as Customer Return, and the adjustment amount with specified price components, such as the principle, shipping, and tax.

You must submit refunds using the adjustment template to issue full refunds or make miscellaneous adjustments for multiple orders simultaneously.

Prepare and upload the adjustments file

To prepare and upload the adjustments file:

1. Download and save the adjustments template.
2. Open the adjustments template and review the **Instructions** and **Definitions** tabs. This template has validation macros that can help you complete the template.
3. Select the **Adjustments** tab in the excel file.

NOTIFICATIONS API

Notifications API

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification

ORDERS API

Orders API

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API

PRODUCT PRICING API

Product Pricing API

Product Pricing API and Notifications FAQ

4. Enter the information for each order you are refunding:
    1. You must populate the `order-id`, `order-item-id`, `adjustment-reason-code`, `currency`, and at least one field ending in `-adj`.
    2. If you are not offering promotions, delete the promotion fields from the file.
    3. Hyphens in order numbers (`order-id`) are required. The feed will not process successfully without hyphens.
    4. Format number columns as text to prevent Excel from removing leading zeros.
    5. Format date columns as `YYYY-MM-DD`.
5. Save the file as a text (`.txt`) file.
6. Follow the Tutorial: Submit a feed to upload the tab delimited file and create a feed.

Input fields

Field name	Definition	Accepted Values	Example
`order-id`	Amazon's unique, displayable identifier for an order, provided in your `OrderReport`.	Alphanumeric text formatted as `##-#####-#######`.	`123-1234567-1234567`
`order-item-id`	Amazon's unique, displayable identifier for an item within an order, provided in your `OrderReport`.	Alphanumeric text, 14 characters.	`12345678901234`
`adjustment-reason-code`	The reason for the refund or adjustment.	- `NoInventory` - `CustomerReturn` - `GeneralAdjustment` - `CouldNotShip` - `DifferentItem` - `Abandoned` - `CustomerCancel` - `PriceError`	`CustomerReturn`
`item-price-adj`	The amount refunded to the buyer for the item. All amounts are aggregates of the quantity, not unit prices. Amounts are	A number with up to 20 digits to the left of the decimal point and two digits required to the right of the	`22.49`

Price Adjustment Automation Workflows Guide
Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API

REPORTS API

Reports API

Report Type Values

Reports JSON Schemas

Reports API FAQ

SALES API

Sales API

SELLER WALLET API

Seller Wallet API

Field name	Definition	Accepted Values	Example
	signed; positive amounts are refunded to the buyer and negative amounts are charged to the buyer (for example, restocking fees). Adjustments can be made within the amount authorized by the customer, but no adjustment can exceed the authorized amount.	decimal point. Do not use commas.	
`shipping-price-adj`	The amount refunded to the buyer for the item's shipment. All amounts are aggregates of the quantity, not unit prices. Amounts are signed; positive amounts are refunded to the buyer and negative amounts are charged to the buyer. Adjustments can be made within the amount authorized by the customer, but no adjustment can exceed the authorized amount.	A number with up to 20 digits to the left of the decimal point and two digits required to the right of the decimal point. Do not use commas.	`4.99`
`item-tax-adj`	The amount refunded to the buyer for the item tax. All amounts are aggregates of the quantity, not unit prices. Amounts are signed; positive amounts are refunded to the buyer and negative amounts are charged to the buyer. Adjustments can be made within the	A number with up to 20 digits to the left of the decimal point and two digits required to the right of the decimal point. Do not use commas.	`2.25`

Field name	Definition	Accepted Values	Example
	amount authorized by the customer, but no adjustment can exceed the authorized amount.		
`shipping-tax-adj`	The amount refunded to the buyer for the item's shipment tax. All amounts are aggregates of the quantity, not unit prices. Amounts are signed; positive amounts are refunded to the buyer and negative amounts are charged to the buyer. Adjustments can be made within the amount authorized by the customer, but no adjustment can exceed the authorized amount.	A number with up to 20 digits to the left of the decimal point and two digits required to the right of the decimal point. Do not use commas.	`0.5`
`gift-wrap-price-adj`	The amount refunded to the buyer for the item's gift wrap tax. All amounts are aggregates of the quantity, not unit prices. Amounts are signed; positive amounts are refunded to the buyer and negative amounts are charged to the buyer. Adjustments can be made within the amount authorized by the customer but, no adjustment can exceed the authorized amount.	A number with up to 20 digits to the left of the decimal point and two digits required to the right of the decimal point. Do not use commas.	`4.99`
`gift-wrap`	The amount refunded to the buyer for the item's gift wrap tax. All	A number with up to 20 digits to the left of the decimal point	`0.45`

TOKENS API

Tokens API

UPLOADS API

Uploads API

VEHICLES API

Vehicles API

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API

Field name	Definition	Accepted Values	Example
`-tax-adj`	amounts are aggregates of the quantity, not unit prices. Amounts are signed; positive amounts are refunded to the buyer and negative amounts are charged to the buyer. Adjustments can be made within the amount authorized by the customer but, no adjustment can exceed the authorized amount.	and two digits required to the right of the decimal point. Do not use commas.	
`item-promotion-adj`	The amount to be refunded for the item's promotion. All amounts are aggregates of the quantity, not unit prices. Amounts are signed; positive amounts are refunded to the buyer and negative amounts are charged to the buyer. Adjustments can be made within the amount authorized by the customer, but no adjustment can exceed the authorized amount.	A number with up to 20 digits to the left of the decimal point and two digits required to the right of the decimal point. Do not use commas.	`2.25`
`item-promotion-id`	Merchant-specified promotion ID.	Alphanumeric text, up to 80 characters.	`FREESHIPHOLIDAY`
`ship-promotion-adj`	The amount to be refunded for the item's shipping promotion. All amounts are aggregates of the quantity, not unit prices. Amounts are signed; positive	A number with up to 20 digits to the left of the decimal point and two digits required to the right of the decimal point.	`1.75`

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API

VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API

Vendor Retail Procurement Transaction Status API

Powered by readme

Field name	Definition	Accepted Values	Example
	amounts are refunded to the buyer and negative amounts are charged to the buyer. Adjustments can be made within the amount authorized by the customer, but no adjustment can exceed the authorized amount.	Do not use commas.	
`ship-promotion-id`	Merchant-specified promotion ID.	Alphanumeric text, up to 80 characters.	`DOLLAR0FFPR0M0`
`quantity-cancelled`	Quantity cancelled of the order-item being adjusted.	A positive integer.	`1`
`currency`	The currency code for the adjustments. This must be the same as the currency code for the adjusted order.	- `USD` - `GBP` - `EUR` - `JPY` - `CAD`	`USD`

Processing report

Follow Step 6. Get information for retrieving the feed processing report to retrieve the processing report for the uploaded feed.

Check the feed processing report for errors. If there are no errors, your feed submission is complete. If there are errors, correct them and submit the corrected feed, starting at Step 1. Create a feed document. Repeat this process until there are no errors in the feed processing report.

When there are errors, the processing report includes details such as:

- **Error code**: A specific code that identifies the type of error.
- **Error message**: A descriptive message explaining the error.
- **Timestamp**: When the error occurred.
- **Affected items**: Details about which parts of the feed or specific items caused the error.
- **Suggested actions**: Guidance on how to correct the error.

Feed platform validation

When the feed platform receives the order adjustment feed, it performs an initial validation to ensure the feed is correctly

formatted and complete before forwarding it to the refund feed processor. If there is an issue (missing required fields or incorrect format), the feed platform reports the issue in the processing report and returns it.

The refund feed processor checks that the feed adheres to the expected structure and syntax. This includes:

- Ensuring all required fields are present.
- Checking that data types (such as numbers and dates) are correct.
- Verifying that field values meet predefined criteria (for example, valid order IDs).

Beyond format, the processor checks if the orders in the feed are eligible for a refund. This involves:

- Ensuring the orders exist and are within the refundable time frame.
- Checking for any conditions that might disqualify an order from being refunded. This includes cases where the item is non-refundable or already refunded.

Any validations the feed fails are reported in the processing report.

Finally, the validator ensures that the refund request is valid and meets all criteria before processing. It includes a thorough review of all aspects of the feed to prevent errors in the refund system.

You can poll the system for the status of the feed you submit to confirm processing or get details of any errors. You use the information in the processing report to correct issues and resubmit the feed. By implementing these validation steps and error handling mechanisms, the system ensures that only correctly formatted and eligible refund requests are processed, while providing clear feedback to you to resolve any issues promptly.

Different types of errors

Error	Possible reasons	Possible solutions
Error attempting to issue refund for order	Invalid `order-id` or `order-item-id` field	Check the order. The order you're trying to refund might not exist in the system, or it might be in an invalid state (for example, already refunded, cancelled, or completed).
Invalid currency code	Invalid currency field	Ensure that the currency code is properly formatted.

Error	Possible reasons	Possible solutions
Missing pricing currency unit	Missing currency field	Double check the currency field provided in the template.
Invalid amount issued for refund item	Invalid `shipping-price-adj`, `shipping-tax-adj`, or `ship-promotion-adj` values in the template	There is an issue with the amount specified for refunding a shipping-related item in an order. Verify the `shipping-price-adj`, `shipping-tax-adj`, and `ship-promotion-adj` values in the template.
base component is only provided and tax is missing for component Shipping	Invalid `item-tax-adj` or `shipping-tax-adj` field	There is an issue with the tax calculation for a shipping component in your system. `item-tax-adj` or `shipping-tax-adj` is missing in the template. Verify these fields.
Order ID missing for refund	Missing `order-id` field	Verify the `order-id` field.
Invalid amount issued for promotion	Invalid `item-promotion-adj`, `item-promotion-id`, `ship-promotion-adj`, or `ship-promotion-id` field	The fields related to price and promotion might be invalid. Re-verify these fields.
Invalid `OrderID` or `AmazonOrderID`	Invalid `order-id` field	Verify the `order-id`.
Could not issue refund because there are several returns associated to return item in order	The provided order is invalid for refund as there are multiple returns associated with this order	Re-verify the order.
Invalid amount issued for refund item for base component (?!Shipping)	Invalid `shipping-price-adj` or `shipping-tax-adj`	Re-verify the `shipping-price-adj` and `shipping-tax-adj` fields in the template.

Note

A refund can still be denied after a successful submission if certain issues arise, such as buyer abuse or the use of an invalid payment method.

Best practices

For best practices using the Feeds API, refer to Feeds API Best Practices.

Updated 19 days ago

← Feeds API

Feed Type Values →